Facultad de Defensa Nacional

Licenciatura en CIBERDEFENSA

Lenguajes de Programación

Programación Orientada a Objetos en PHP

v 1.0

Docentes:

Ing. Sebastián Balestrini

Esp. Ing. Gustavo Di Stefano

Carga horaria: 64 hs

Tabla de contenido

Tabla de contenido 1

Programación Orientada a Objetos en PHP 2

Instanciación 2

Métodos y propiedades 2

Constructores y destructores 4

Uso de $this y contexto de objeto 5

Herencia en PHP 5

Sobreescritura de métodos 6

Polimorfismo aplicado en PHP 7

Palabra clave parent 8

Clases abstractas 9

Interfaces 10

Programación Orientada a Objetos en PHP

En PHP, al igual que en otros lenguajes orientados a objetos, una clase se define utilizando la palabra clave **class**. Dentro de la clase se declaran las propiedades (variables que describen el estado) y los métodos (funciones que definen el comportamiento).

```php
class Persona {
    // Propiedades
    public $nombre;
    public $edad;

    // Métodos
    public function saludar() {
        echo "Hola, mi nombre es " . $this->nombre;
    }
}
```

En PHP es una buena práctica que cada clase se guarde en un archivo separado con el mismo nombre de la clase (por ejemplo, Persona.php o persona.class.php). Esto facilita la organización y la reutilización del código.

Instanciación

Una vez que tenemos definida una clase, podemos crear objetos (instancias) a partir de ella. A este proceso se lo llama instanciación. En PHP, los objetos se crean utilizando la palabra clave new, seguida del nombre de la clase.

```php
// Definición de la clase
class Persona {
    ///...
}

// Creación de objetos
$persona1 = new Persona();
```

Métodos y propiedades

En PHP, las propiedades son las variables que pertenecen a una clase, y los métodos son las funciones definidas dentro de esa clase. La forma de declarar ambos es muy parecida a lo visto en Java, pero con la sintaxis propia de PHP.
Propiedades

- Se definen dentro de la clase, utilizando un modificador de acceso (public, private, protected).
- Siempre se preceden con el símbolo $, ya que en PHP todas las variables lo requieren

```php
// Definición de la clase
class Persona {
    public $nombre;
    public $edad;
    private $legajo;

}
```

Vayamos ahora a un ejemplo completo:

```php
// Definición de la clase
class Persona {
    public $nombre;
    public $edad;
    private $legajo;

    public function saludar() {
        echo "Hola, mi nombre es " . $this->nombre;
    }
}

// Creación de objetos
$persona1 = new Persona();
$persona1->nombre = "Ana";
$persona1->edad = 25;

$persona2 = new Persona();
$persona2->nombre = "Juan";
$persona2->edad = 30;

// Uso de métodos
$persona1->saludar(); // Muestra: Hola, mi nombre es Ana
$persona2->saludar(); // Muestra: Hola, mi nombre es Juan
```

Aquí vemos que $persona1 y $persona2 son objetos distintos creados a partir de la misma clase Persona pero cada uno tiene sus propios valores de $nombre y $edad, es decir, mantienen su propio estado. También vemos que ambos pueden ejecutar el método saludar(), pero el resultado depende de los datos de cada objeto.
Esto refleja la esencia de la POO: una clase como molde, múltiples objetos como instancias concretas.
También puede observarse que al ser públicos nombre y edad pueden ser accedidos mientras que, $legajo es privado y solo podría ser accedido desde un método de la clase como por ejemplo:

```php
// Definición de la clase
class Persona {
    public $nombre;
    public $edad;
    private $legajo;

    public function saludar() {
        echo "Hola, mi nombre es " . $this->nombre;
    }

    public function setLegajo($legajo) {
        $this->legajo = $legajo;
    }
    public function getLegajo() {
        return $this->legajo;
    }

}
```

Esto es un claro ejemplo de encapsulamiento implementado a través de las definiciones de acceso (Público, privado y protegido) definiendo el nivel de visibilidad de atributos y métodos.

- public: accesible desde cualquier parte del código.
- private: accesible únicamente dentro de la clase donde se declaró.
- protected: accesible desde la propia clase y también desde las clases que heredan de ella.

Constructores y destructores

En POO, los constructores son métodos especiales que se ejecutan automáticamente al crear un objeto. Su propósito principal es inicializar las propiedades de la clase para que el objeto comience en un estado válido.
En PHP, el constructor se define con el método mágico __construct().

```php
class Persona {
    public $nombre;
    public $edad;

    // Constructor
    public function __construct($nombre, $edad) {
        $this->nombre = $nombre;
        $this->edad = $edad;
    }

    public function saludar() {
        echo "Hola, me llamo " . $this->nombre . " y tengo " . $this->edad . " años.";
    }
}

// Crear objetos con constructor
$persona1 = new Persona("Ana", 25);
$persona2 = new Persona("Luis", 30);

$persona1->saludar(); // Hola, me llamo Ana y tengo 25 años.
$persona2->saludar(); // Hola, me llamo Luis y tengo 30 años.
```

Los destructores son métodos especiales que se ejecutan cuando un objeto deja de existir o se libera de la memoria. En PHP se definen con __destruct().
Aunque se usan menos que los constructores, pueden ser útiles, por ejemplo, para cerrar conexiones a bases de datos o liberar recursos.

```php
class Conexion {
    public function __construct() {
        echo "Conexión iniciada<br>";
    }

    public function __destruct() {
        echo "Conexión cerrada<br>";
    }
}

$conexion = new Conexion();
// Cuando el script finaliza o se libera el objeto, se llama a __destruct()
```

Uso de $this y contexto de objeto

En PHP, la palabra clave $this es fundamental cuando trabajamos con objetos. Representa una referencia al propio objeto desde dentro de la clase. Dicho de otra manera, $this señala “este objeto en particular” que está ejecutando un método en un momento dado.
Cada vez que un objeto llama a un método, dentro de ese método podemos usar **$this** para:

- Acceder a las propiedades del objeto.
- Invocar a otros métodos de la misma clase.

```php
class Persona {
    public $nombre;
    public $edad;

    public function presentarse() {
        echo "Hola, soy " . $this->nombre . " y tengo " . $this->edad . " años.";
    }

    public function cumplirAnios() {
        $this->edad++;
    }
}

// Crear un objeto
$persona1 = new Persona();
$persona1->nombre = "Ana";
$persona1->edad = 25;

$persona1->presentarse(); // Hola, soy Ana y tengo 25 años.
$persona1->cumplirAnios();
$persona1->presentarse(); // Hola, soy Ana y tengo 26 años.
```

Herencia en PHP

La herencia en PHP funciona de manera muy similar a otros lenguajes orientados a objetos como Java. Permite que una clase (llamada clase hija o subclase) reutilice propiedades y métodos de otra clase (llamada clase padre o superclase).
En PHP se utiliza la palabra clave extends para indicar herencia.

```php
class Persona {
    public $nombre;
    public $edad;

    public function presentarse() {
        echo "Hola, soy " . $this->nombre;
    }
}

class Estudiante extends Persona {
    public $carrera;

    public function estudiar() {
        echo $this->nombre . " está estudiando " . $this->carrera;
    }
}

// Uso
$estudiante = new Estudiante();
$estudiante->nombre = "Ana";
$estudiante->carrera = "TUTI";
$estudiante->presentarse(); // Hola, soy Ana
$estudiante->estudiar();    // Ana está estudiando TUTI
```

Aquí, la clase Estudiante hereda de Persona, por lo que además de tener su propio método estudiar(), puede utilizar el método presentarse() definido en la clase padre.

La herencia permite construir jerarquías de clases, donde las clases hijas pueden especializar el comportamiento de las clases padre.

Sobreescritura de métodos

La **sobrescritura de métodos** (method overriding) ocurre cuando una clase hija redefine un método que ya fue declarado en la clase padre. Esto permite modificar o especializar el comportamiento heredado sin alterar el código de la clase original.
En PHP, para sobrescribir un método basta con declararlo nuevamente en la clase hija con el mismo nombre. Cuando se llama a ese método desde un objeto de la subclase, se ejecutará la versión redefinida.

```php
class Persona {
    public $nombre;

    public function presentarse() {
        echo "Hola, soy " . $this->nombre;
    }
}
class Estudiante extends Persona {
    public $carrera;

    // Sobrescritura del método presentarse()
    public function presentarse() {
        echo "Hola, soy " . $this->nombre . " y estudio " . $this->carrera;
    }
}

// Uso
$estudiante = new Estudiante();
$estudiante->nombre = "Ana";
$estudiante->carrera = "TUTI";
$estudiante->presentarse();
// Muestra: Hola, soy Ana y estudio TUTI
```

En este ejemplo, el método presentarse() existe en ambas clases.

- Si llamamos al método desde un objeto Persona, se ejecuta la versión original.
- Si lo hacemos desde un Estudiante, se ejecuta la versión sobrescrita.

Esto es útil para mantener una interfaz común (métodos con el mismo nombre) pero con comportamientos diferentes en cada subclase.

Polimorfismo aplicado en PHP

El **polimorfismo** es la capacidad que tienen los objetos de diferentes clases de responder a un mismo mensaje (método) de manera distinta. En otras palabras: un mismo método puede tener múltiples formas de ejecutarse, dependiendo de la clase del objeto que lo invoque.

En **PHP**, el **polimorfismo** se logra principalmente a través de:

- Herencia y sobrescritura de métodos, donde cada subclase implementa su propia versión de un método heredado.

- Interfaces y clases abstractas (que veremos más adelante), donde diferentes clases se comprometen a implementar los mismos métodos pero con lógicas distintas.

```php
class Persona {
    public $nombre;

    public function presentarse() {
        echo "Hola, soy " . $this->nombre;
    }
}

class Estudiante extends Persona {
    public function presentarse() {
        echo "Hola, soy estudiante y me llamo " . $this->nombre;
    }
}

class Profesor extends Persona {
    public function presentarse() {
        echo "Buenos días, soy el profesor " . $this->nombre;
    }
}
```

Y el uso podría ser algo como esto:

```php
// Uso
$personas = [
    new Estudiante(),
    new Profesor()
];

$personas[0]->nombre = "Ana";
$personas[1]->nombre = "Sebastián";

foreach ($personas as $p) {
    $p->presentarse();
    echo "<br>";
}
```

"Aunque ambos objetos (Estudiante y Profesor) se guardan en el mismo arreglo como Persona, cada uno ejecuta su propia versión de presentarse(). Esto ilustra claramente el polimorfismo: un mismo mensaje, diferentes respuestas."

Palabra clave parent

En PHP, la palabra clave **parent** se utiliza dentro de una clase hija para hacer referencia a la clase padre. Es especialmente útil en dos casos:

1. Llamar a métodos de la clase padre que han sido sobrescritos en la clase hija.
2. Invocar el constructor de la clase padre para reutilizar su lógica de inicialización.

De esta forma, **parent** permite extender el comportamiento heredado sin perder lo que ya estaba definido en la superclase.

```php
class Persona {
    public $nombre;

    public function __construct($nombre) {
        $this->nombre = $nombre;
    }
}

class Estudiante extends Persona {
    public $carrera;

    public function __construct($nombre, $carrera) {
        // Reutiliza el constructor de Persona
        parent::__construct($nombre);
        $this->carrera = $carrera;
    }
}

$estudiante = new Estudiante("Ana", "TUTI");
echo $estudiante->nombre;  // Ana
```

Ahora bien, analicemos el siguiente ejemplo:

```php
class Persona {
    public function presentarse() {
        echo "Hola, soy una persona.";
    }
}

class Profesor extends Persona {
    public function presentarse() {
        // Llamada al método original de Persona
        parent::presentarse();
        echo " Soy profesor de programación.";
    }
}

$prof = new Profesor();
$prof->presentarse();
// Muestra: Hola, soy una persona. Soy profesor de programación
```

En este caso, Profesor sobrescribe el método presentarse(), pero antes de añadir su mensaje personalizado, invoca al método original con parent::presentarse().

Clases abstractas

Una clase abstracta es una clase que no puede ser instanciada directamente, sino que sirve como modelo para que otras clases hereden de ella. Puede contener:

- Métodos con implementación completa.
- Métodos abstractos, es decir, métodos definidos solo en su firma (nombre y parámetros), pero sin código.

Las clases hijas están obligadas a implementar los métodos abstractos para poder implementarse

```php
abstract class Persona {
    public $nombre;

    public function __construct($nombre) {
        $this->nombre = $nombre;
    }
    // Método abstracto (sin cuerpo)
    abstract public function presentarse();
}
class Estudiante extends Persona {
    public function presentarse() {
        echo "Hola, soy estudiante y me llamo " . $this->nombre;
    }
}

$est = new Estudiante("Ana");
$est->presentarse();
// Hola, soy estudiante y me llamo Ana
```

Interfaces

Una interfaz es un conjunto de métodos abstractos que especifica un contrato que las clases deben cumplir.
A diferencia de las clases abstractas:

- Una interfaz no contiene código de implementación, solo definiciones de métodos.
- Una clase puede implementar múltiples interfaces (PHP permite herencia simple, pero múltiples implementaciones de interfaces).

```php
interface Notificable {
    public function enviarNotificacion($mensaje);
}

class Usuario implements Notificable {
    public $email;

    public function __construct($email) {
        $this->email = $email;
    }

    public function enviarNotificacion($mensaje) {
        echo "Enviando a " . $this->email . ": " . $mensaje;
    }
}

$user = new Usuario("ana@example.com");
$user->enviarNotificacion("Bienvenida!");
```

Aquí, la clase Usuario implementa la interfaz Notificable, lo que garantiza que tenga un método enviarNotificacion().